RGB-D Fusion

QianJian

Outlines

I. Purpose

II. Datasets

III. Algorithms

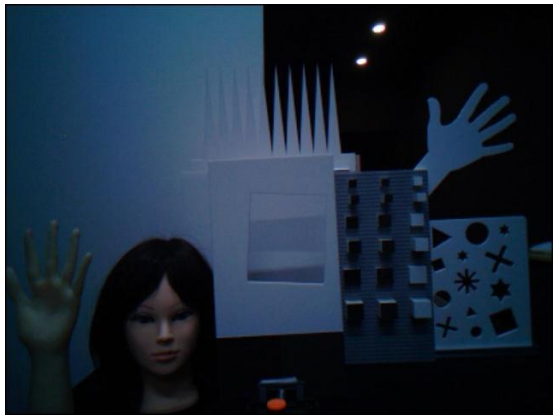
IV. Compare

V. Research

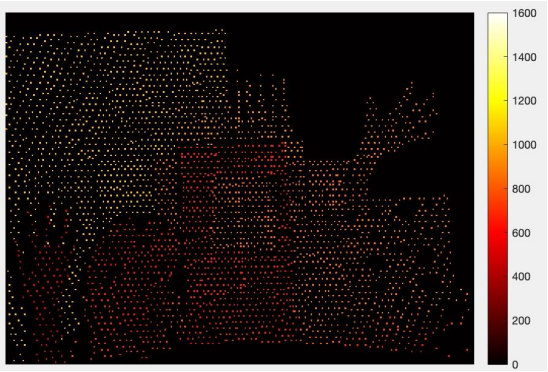
VI. Conclusion

I. Purpose

RGB



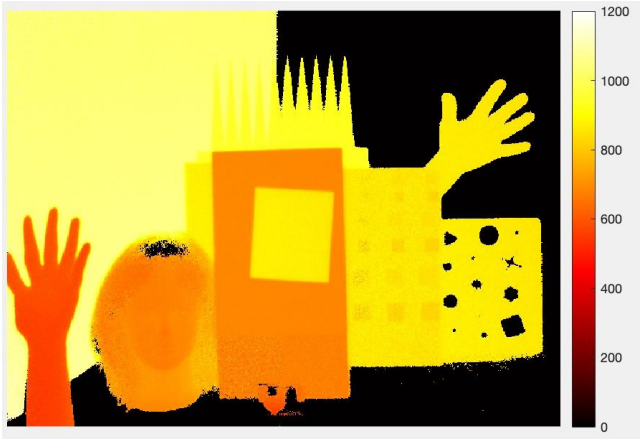
Sparse Depth



Ho
w? ?

Fusion

Dense Depth



ToF cameras generally provide

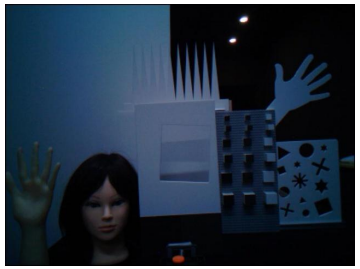
1. Low resolution
2. Sparse depth

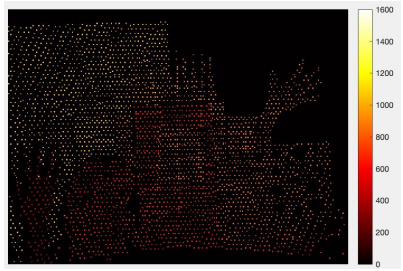
Various CV applications need dense dep

1. Autonomous driving
2. AR/VR
3. 3D scene reconstruction
4. SLAM

II. Datasets

1. Spot-ToF data

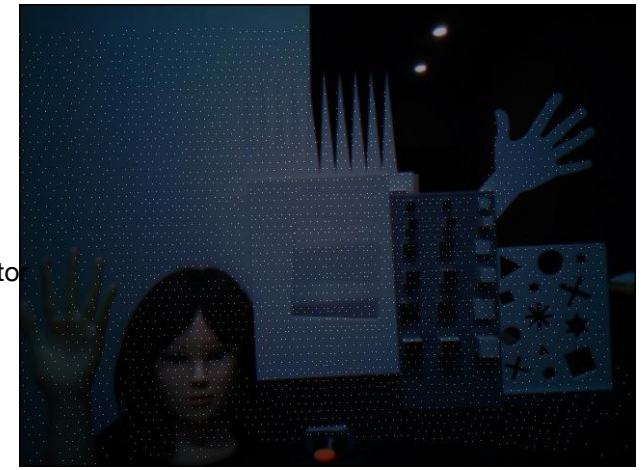




HW: BKF518
Frames: RGB--820x616
Depth--160x120

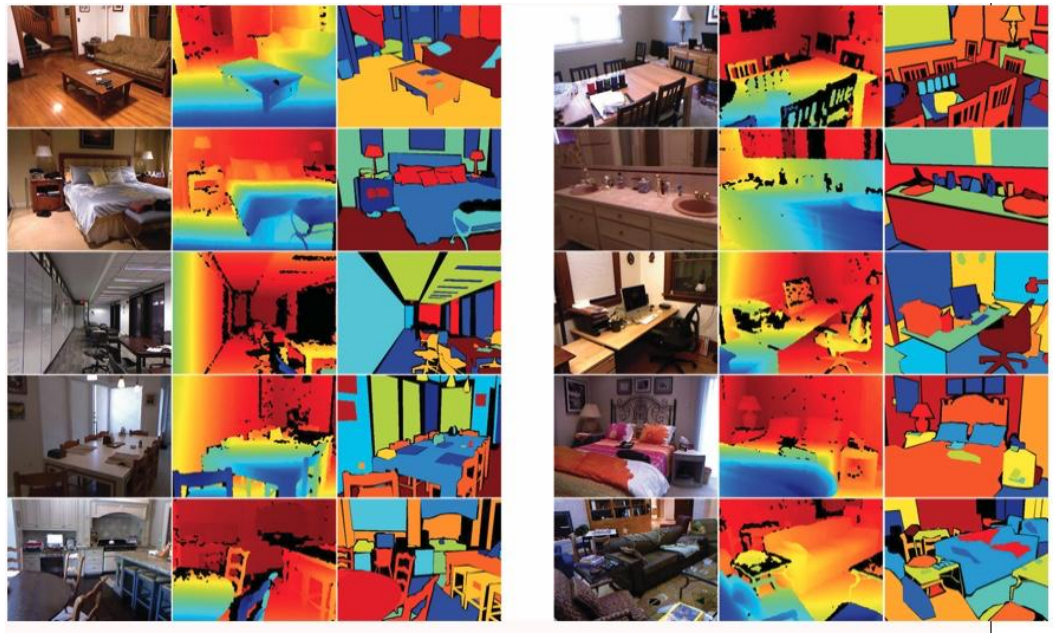
Preprocessing:

1. Alignment between depth data and RGB data by using calibration parameters
2. Interpolate depth image and RGB image with same size



Intrisic parameters: fx, fy, cx, cy
Homogeneous transformation: rotation matrix , translation vecto
Distortion parameters are k1, k2, k3, p1 and p2

2.NYU-Depth-v2 dataset



HW: Microsoft Kinect
Original frames :640x480
Training Data frames: 304x228

Link: https://cs.nyu.edu/~silberman/datasets/nyu_depth_v2.html

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

```
x' = x + x*(K1*r^2 + K2*r^4 + K3*r^6) + P1*(r^2 + 2*x^2) + 2*P2*x*y
y' = y + y*(K1*r^2 + K2*r^4 + K3*r^6) + P2*(r^2 + 2*y^2) + 2*P1*x*y
```

III. Algorithms

1. joint bilateral interpolation

$$\tilde{d}_p = \begin{cases} d_p & \text{if } d_p! = 0 \\ \frac{1}{W}\sum_{i=1}^{n} (d_q f(\|p - q_i\|) g(\|\tilde{I}_p - \tilde{I}_{q_i}\|)) & \text{if } d_p = 0 \end{cases}$$

```
        if not np.isnan(source_upsampled[value_y, value_x]):
            weight = kernel_range * kernel_spatial
            patch_source_upsampled = source_upsampled[value_y, value_x]
            curr_pixel += patch_source_upsampled * weight
            k_p += weight
if not np.isnan(source_upsampled[y,x]):
    result[y - padding,x - padding] = source_upsampled[y, x]
elif k_p ==0:
    result[y - padding, x - padding] = source_upsampled[y, x]
else:
    result[y - padding,x - padding] = np.round(curr_pixel / k_p)
```

$\tilde{I}$: high resolution original RGB data
d : sparse depth data
$\tilde{d}_p$: estimated depth data at pixel p
f : the spatial Gaussian kernel, defining the Euclidean distance between pixels
g : the range filter kernel, defining the pixel intensity value difference between pixels by using guidance image $\tilde{I}$
W: a normalizing factor
p, q : coordinates of pixels

Paremeters for tuning:

radius : windows size surrounding the processed pixel, should be an odd value
sigma_spatial : the standard deviation of spatial interpolation kernel , The larger the value, the larger the local influence spatial
sigma_range : the standard deviation of range interpolation kernel, The larger the value, the larger the local influence range

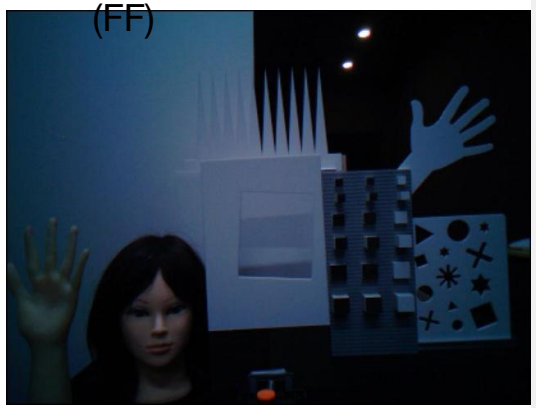
III. Algorithms

1. joint bilateral interpolation

Sparse depth with invalid pixels

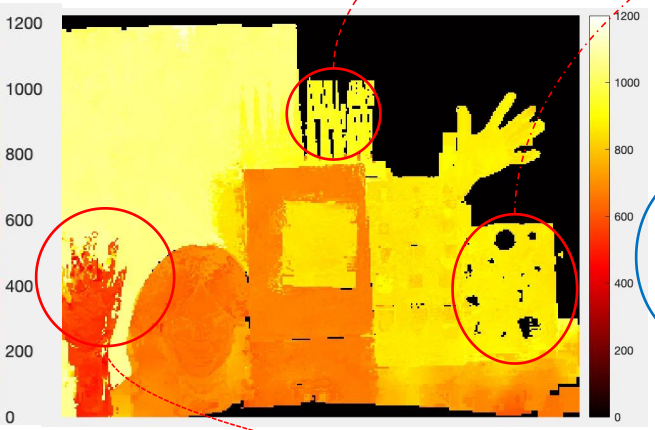
Those holes recovered not well

RGB Data (FF)

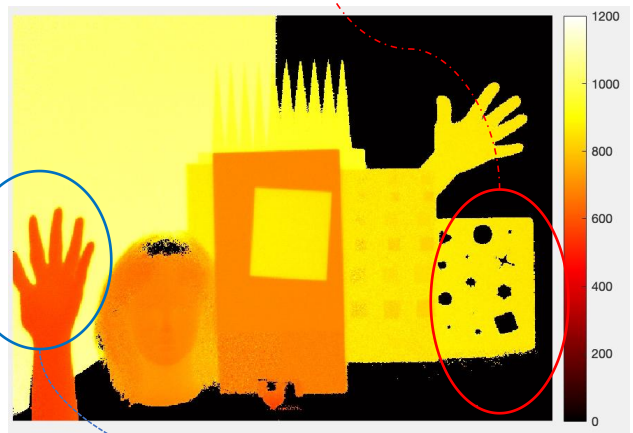


Sparse depth Data

Output of JBI

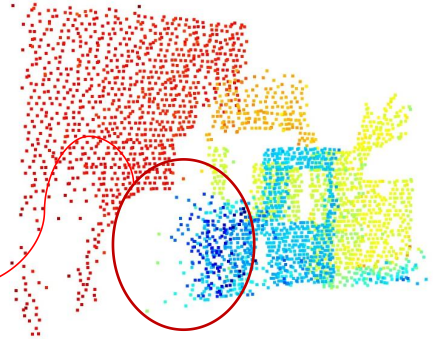


Full Field



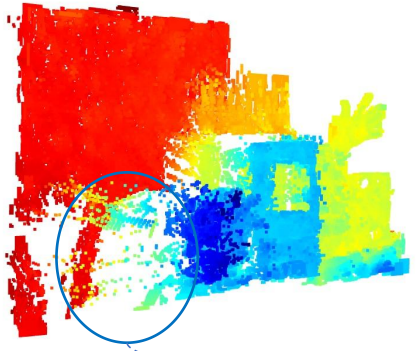
blurry

Sparse depth point cloud

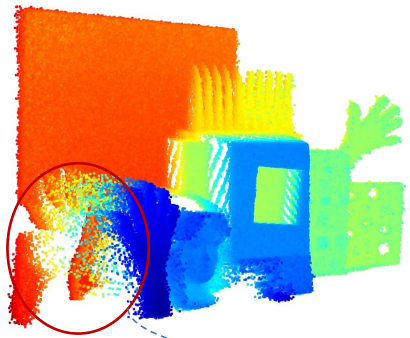


Invalid point cloud

Output of JBI point cloud



FF point cloud



Flying pixel is better than GT

III. Algorithms

2. Anisotropic Total Generalized Variation

$$\min_{u,v} \{ \boxed{a_1 \int_{\Omega_H} |T^{\frac{1}{2}}(\nabla u - v)|\, dx + a_0 \int_{\Omega_H} |\nabla v| dx} + \boxed{\int_{\Omega_H} w|(u - D_S)|^2 dx} \}$$

Regularization term

Data term

$$T^{\frac{1}{2}} = \boxed{\exp(-\beta|\nabla I_H|^{\gamma})} nn^T + n^{\perp} n^{\perp T}$$

anisotropic diffusion factor

D_S : sparse depth data

u : estimated dense depth data

w : a weighting operator, zero at unmapped image points, between zero and one on the basic points

$T^{\frac{1}{2}}$: an anisotropic diffusion tensor

I_H: high resolution original RGB data

β, γ : scalars to adjust the magnitude and the sharpness of the tensor

n : normalized direction of image gradient

$n^{\perp}$: the normal vector to the gradient

a_1, a_0: balance relative weight between data term and regularization term

v : a random auxiliary variables

Solve: primal-dual optimization

step1: updated the dual variables by using gradient ascend

step2: updated the primal variables by using gradient desce

step3: the primal variables are refined in an overrelaxation s

Any iteration n steps are calculated according to:

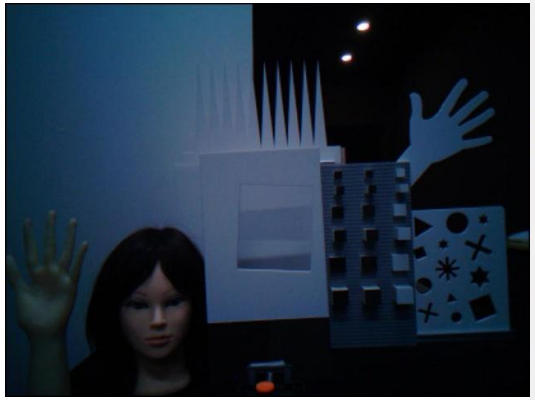
$$\begin{cases} p^{n+1} = \mathbb{P}_p\{p^n + \sigma_p a_1 (T^{1/2}(\nabla \bar{u}^n - \bar{v}^n))\} \\ q^{n+1} = \mathbb{P}_q\{q^n + \sigma_q a_0 \nabla \bar{v}^n\} \\ u^{n+1} = \dfrac{u^n + \tau_u (a_1 \nabla^T T^{1/2} p^{n+1} + w D_S)}{1 + \tau_u w} \\ v^{n+1} = v^n + \tau_v (a_0 \nabla^T q^{n+1} + a_1 T^{1/2} p^{n+1}) \\ \bar{u}^{n+1} = u^{n+1} + \theta(u^{n+1} - \bar{u}^n) \\ \bar{v}^{n+1} = v^{n+1} + \theta(v^{n+1} - \bar{v}^n) \end{cases}$$

III. Algorithms

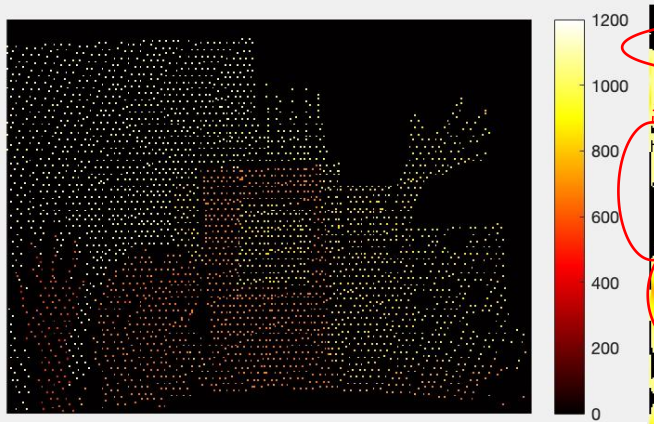
2. Anisotropic Total Generalized Variation

The edage is not recovered completely

RGB Data

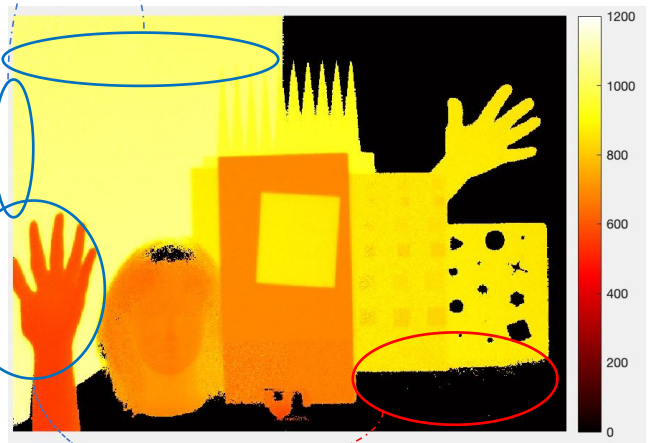


Sparse depth Data



Output of ATGV

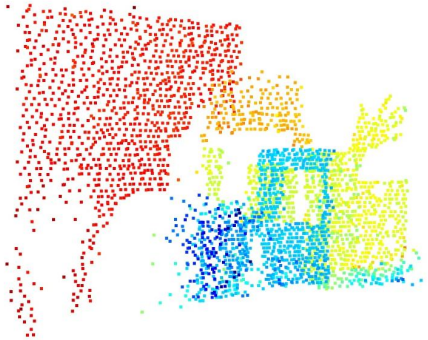
FF



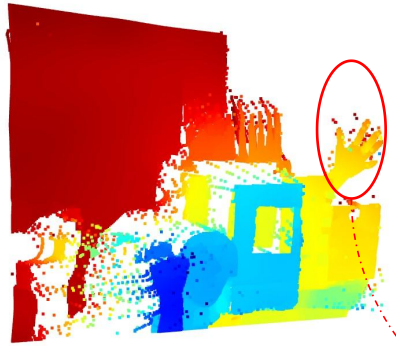
Very blurry

The table recovered well

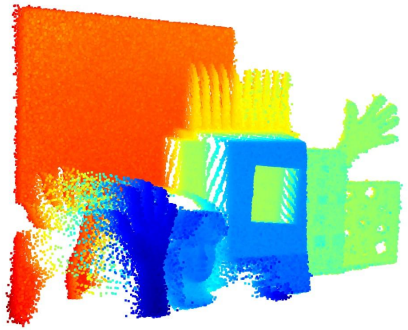
Sparse depth point cloud



Output of ATGV point cloud

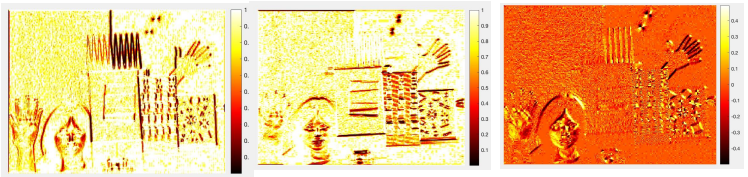


FF point cloud



Flying pixel

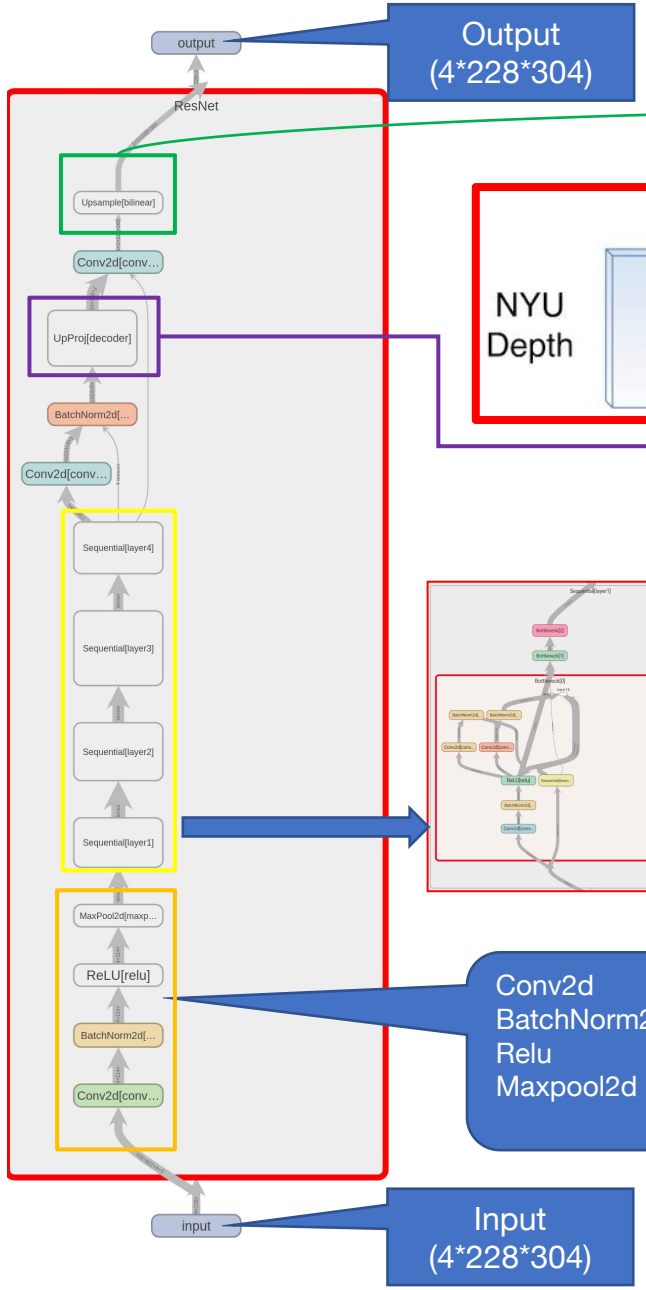
anisotropic diffusion tensor



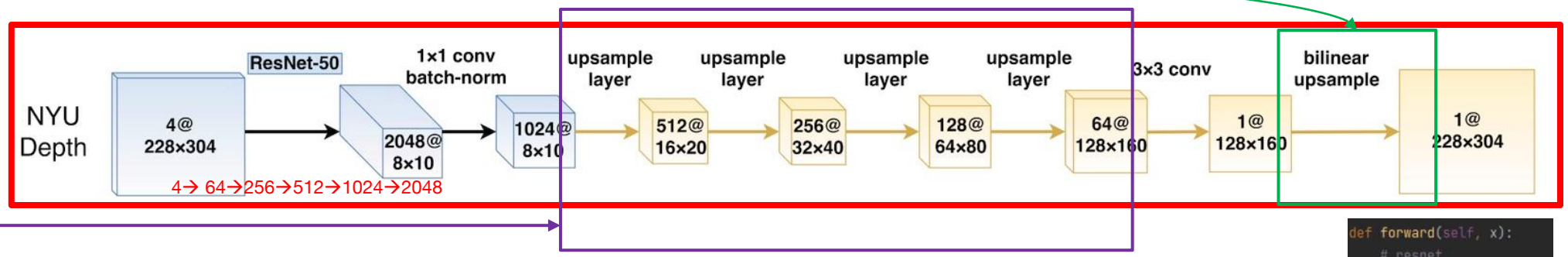
III. Algorithms

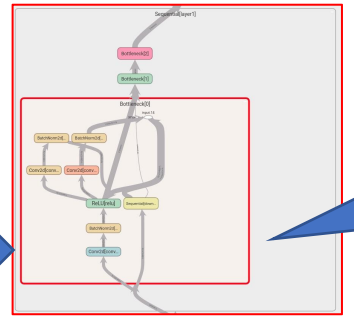
3. ResNet50

CNN architecture for nyu-depth-v2

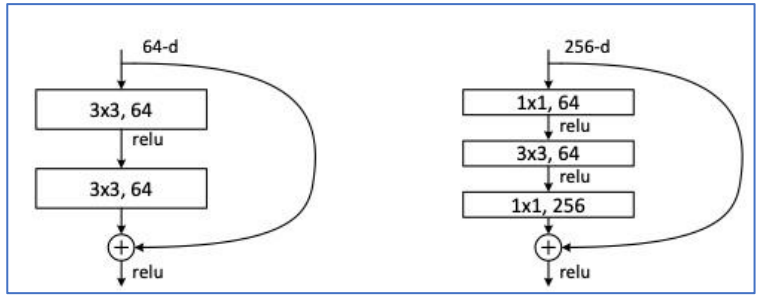


Output
(4*228*304)





Conv2d
BatchNorm2d
Conv2d
BatchNorm2d
Conv2d
BatchNorm2d
Relu



There are two basic cells of Residual module:
The first one is same channels of input and output
The second is different channels of input and output

```
def forward(self, x):
    # resnet
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)

    x = self.conv2(x)
    x = self.bn2(x)

    # decoder
    x = self.decoder(x)
    x = self.conv3(x)
    x = self.bilinear(x)

    return x
```

Conv2d
BatchNorm2d
Relu
Maxpool2d

BatchNorm2d

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

https://blog.csdn.net/qq_29750461

Relu

$$f(x) = \max(0, x)$$

Model : Resnet50
Criteria: Mean squared error
Optimizer: Stochastic gradient desc

```
Total number of parameters in netweorks is 63566144
```

Input
(4*228*304)

III. Algorithms

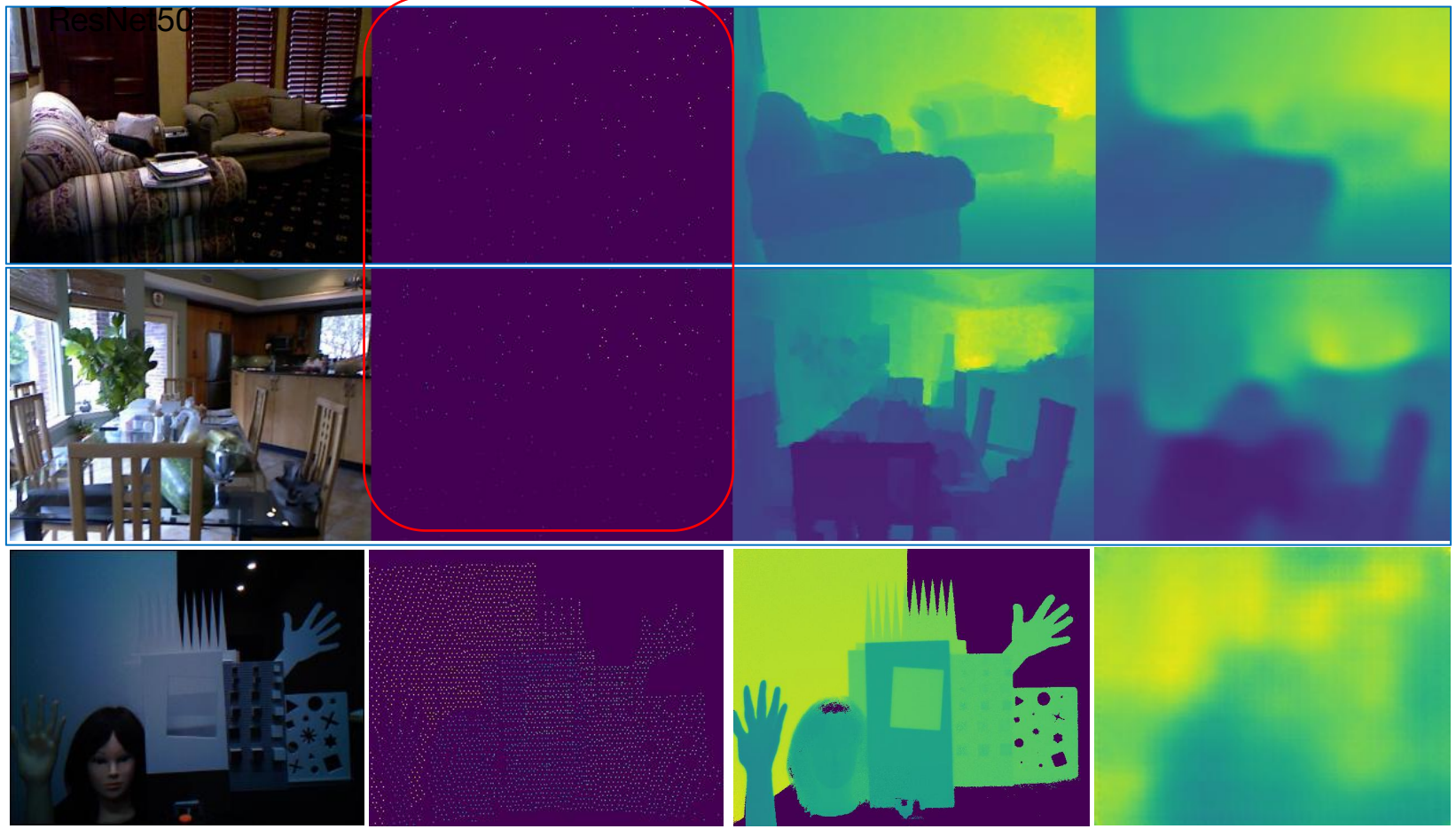
3. ResNet50

Sparse depth data randomly selected from FF, if we increase the dots , the results may be good (haven't tested). Here we follow the sparsity sets in the paper.

RGB Data | Sparse depth Data | FF | Output of ResNet50

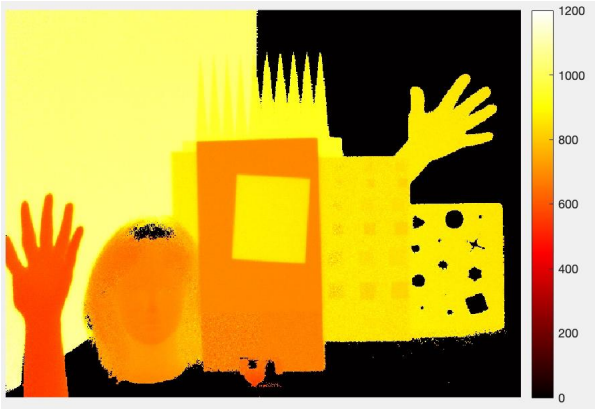
NYU-Depth-v2 dataset

Spot-ToF data



IV. Comparison

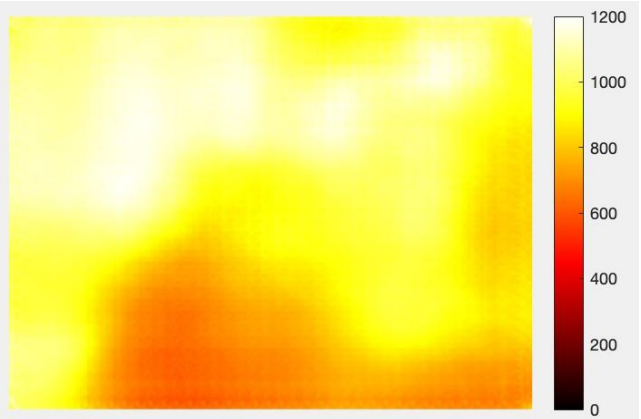
FF



joint bilateral interpolation

Anisotropic Total Generalized Variation

ResNet5



JBI: The edges are sharp, less flying pixels ,more invalid pixels
ATGV: smooth, the plastic hands are very blurry
ResNet50 : Since the model don’t train by Spot-ToF data, the estimated output is not good

V. More Research

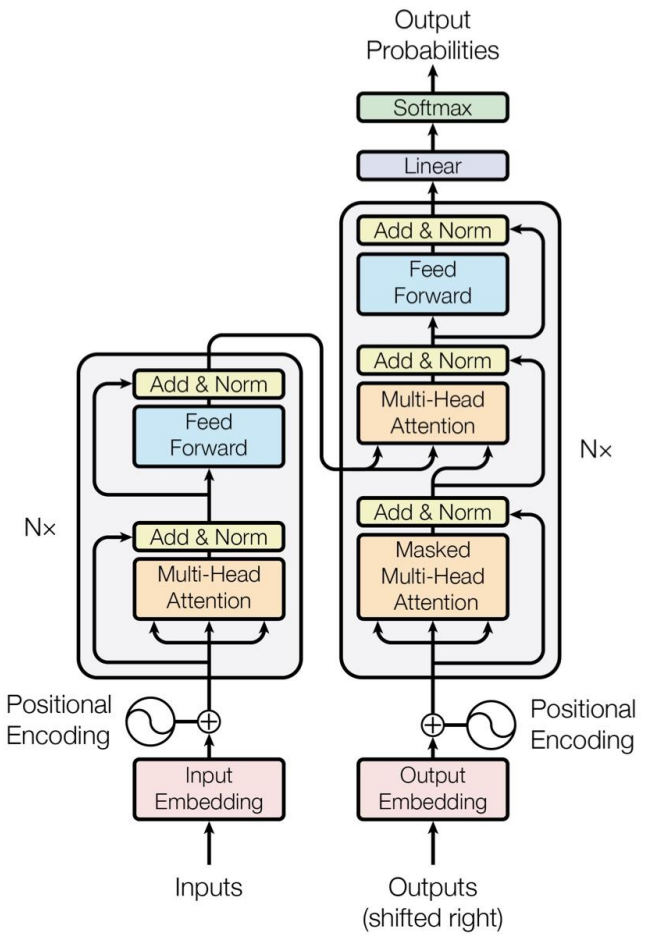


Figure 1: The Transformer - model architecture.

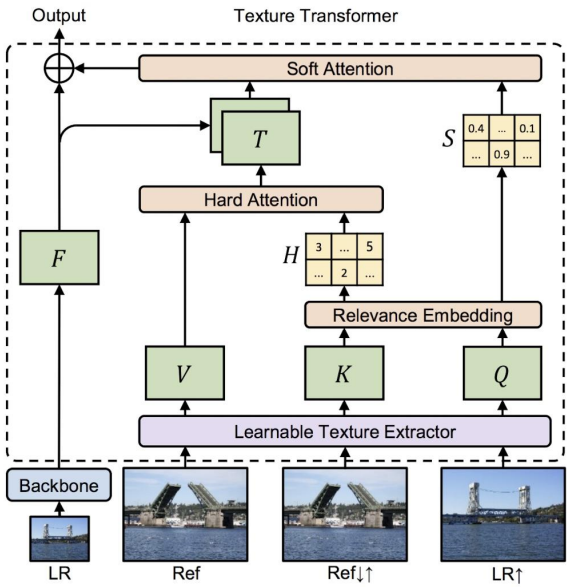
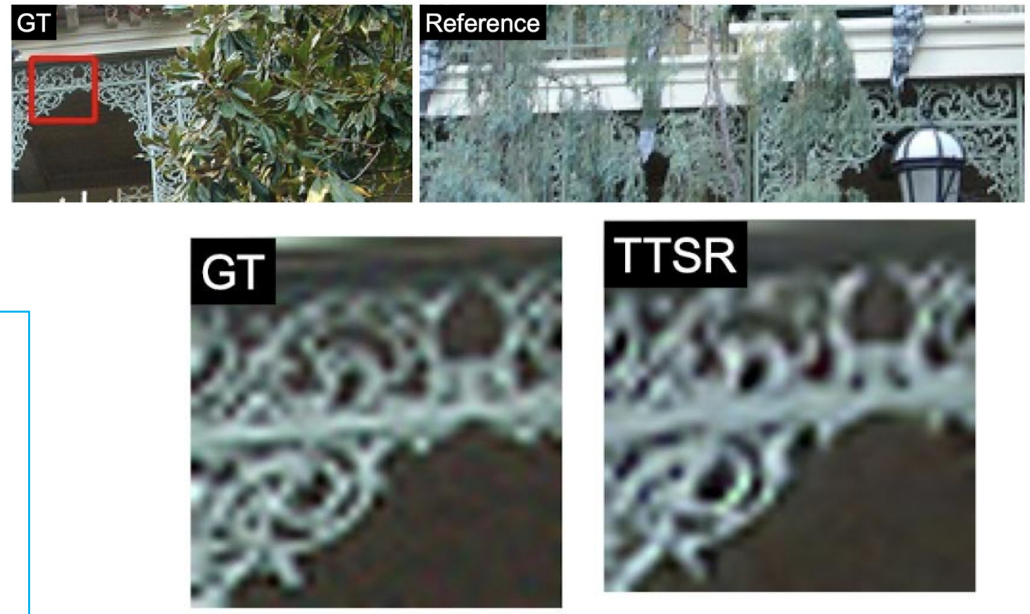


Fig. 11. Diagram of the texture transformer module. Figure is from [14].



VI. Conclusion

1. Before fusion, depth data and RGB data should be alignment at same coordinate with same image size
2. For deep learning method, the training datasets have a significant impact on the output
3. Overall, for smoothing ATGV is better than JBI

Thank you for your attention!